

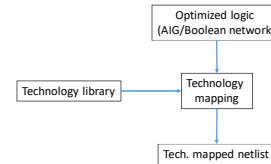
Technology Mapping

Technology Mapping (TM)

TM transforms a multi-level Boolean network into an interconnection of primitive gates from a library.

Goal:

- Minimize the area under the delay constraint
- Minimize the delay under the area constraint



Technology Library

Gate	Cost	Symbol
INV	2	
NAND2	3	
NAND3	4	
NAND4	5	
AOI21	4	
AOI22	5	
XOR	4	

Note: Specific multi-output functions (adders, encoders), as well as registers, may be identified and bound to the corresponding cells using simple replacement rules

A library is a set of cells, each one is characterized by

- A single-output combinational logic function
- An area cost
- The ip/op propagation delay can be a
 - Function of the load, or fanout, for rising/falling transitions at the output
 - Or by worst-case values

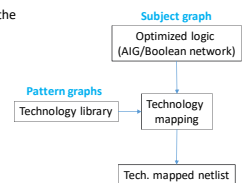
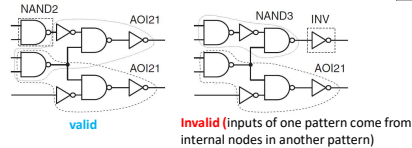
Technology Mapping as a Graph Covering

TM as the optimization problem: find a minimum cost covering of the subject graph by choosing from the collection of pattern graphs for all gates in the library.

A cover is a collection of pattern graphs such that every node of the subject graph is contained in one or more of the pattern graphs.

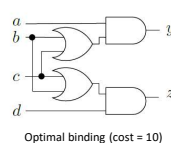
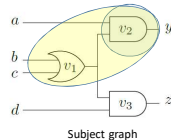
Valid cover: inputs of one pattern in the covering must be the outputs of some other pattern in the covering.

Q: Which cover is valid?



Technology Mapping as a Graph Covering

Library	Cost
AND2	4
OR2	4
OA21	5



- Find all possible matches for all vertices of the subject graph
- Select enough matches to cover all internal vertices

Note: for each selected match, its inputs need to be available as outputs of other matching cells (**valid cover**)

- $v1: (m1 + m4 + m5), v2: (m2 + m4), v3: (m3 + m5)$
- If we bind AND2 to $v2$, we must bound OR2 (only choice) with $v1: m2 \rightarrow m1$

Similarly, $m3 \rightarrow m1$

$(m1 + m4 + m5)(m2 + m4)(m3 + m5)(m2' + m1)(m3' + m1)$ must be satisfiable

Match	vertices
m1	{v1, OR2}
m2	{v2, AND2}
m3	{v3, AND2}
m4	{v1, v2, OA21}
m5	{v1, v3, OA21}

Technology Mapping Algorithms

Steps:

- Matching – identifies possible substitution of library cells for a node in the subject graph
- Covering – optimal selection of matches

How to solve the matching?

- Boolean – library cells and the portion of the subject graph are described as Boolean functions
- Structural – graphs representing algebraic decomposition of Boolean functions are used

Boolean matching: Given two single-output combinational functions $f(x)$ and $g(y)$ with the same number of support variables, we say that f matches g if there exists a permutation matrix P such that $f(x) = g(Px)$

Structural matching: Given a structural representation of two functions by two graphs in a pre-defined format (e.g., networks of two input NAND gates and inverters), there is a structural match if the graphs are isomorphic

Structural matching $\rightarrow$ Boolean matching, but the converse is not true

Boolean and Structural Matching

Boolean matching: Given two single-output combinational functions $f(x)$ and $g(y)$ with the **same** number of support variables, we say that f **matches** g if there exists a permutation matrix P such that $f(x) = g(Px)$

Structural matching: Given a structural representation of two functions by two graphs in a pre-defined format (e.g., networks of two input NAND gates and inverters), there is a structural match if the graphs are isomorphic

Example:

1. $f = ab + c$; $g = p + qr$ Structural match? Yes Boolean match? Yes

2. $f = xy + x'y' + y'z$; $g = xy + x'y' + xz$ Boolean match? Yes Structural match? No

The Boolean matching problem is intractable

The structural matching problem for general functions (represented by DAG) is conjectured to be intractable (transformable into the graph isomorphism problem)

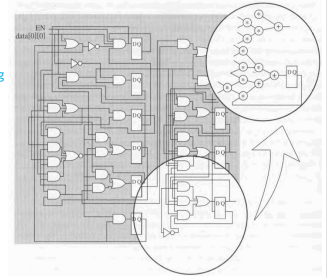
Coping with the Complexity of TM

Major challenge in TM: Covering

To make the covering tractable most heuristic TM algorithms apply two pre-processing steps to the network before covering: **decomposition and partitioning**

Decomposition:

- **Goal:** express all local functions using base functions (e.g., NAND and NOT)
- Guarantee a solution to the covering by ensuring that each vertex is covered by at least one match (e.g., trivial binding)
- Decompositions into base functions are not unique, therefore, affect the quality of the solution



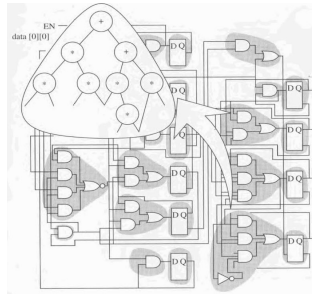
Network decomposition into base functions (two-input ORs and two input ANDs)

Partitioning

- Isolates the combinational portion of a network from the sequential portion and from the IOs
- Allows the covering algorithm to consider a collection of multi-input single-output networks in place of a multi-input multi-output network

Rationale:

- size of the covering problem becomes smaller
- Covering problem becomes tractable (with additional assumptions)



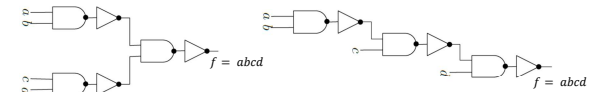
Network partitioning into subject graphs

Structural Matching

Structural matching performs pattern matching between subject graph and library cells (without considering Boolean functionality)

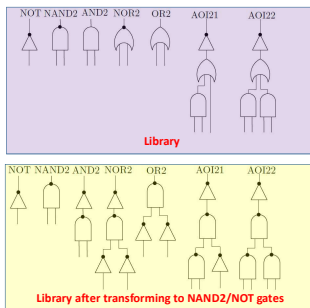
Both the subject graph and library cells need to be decomposed in terms of same base functions

Note: Some library cells may have more than one corresponding pattern graphs (e.g., $f = abcd$)

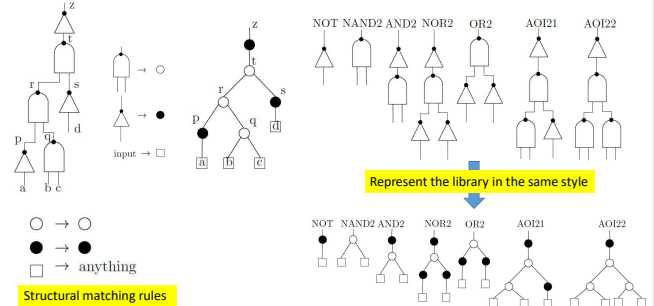


Assumption: We decompose library cells into trees or leaf DAGs (paths from the root converge only at leaves)

Tree Covering: Library Representation



Representation for Matching

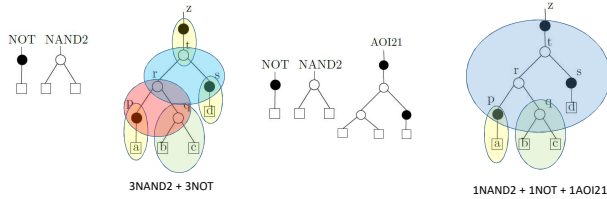


Structural matching rules

Note: Need to account for symmetry during matching (e.g., AOI21 can be matched in 2 ways)

Tree Covering

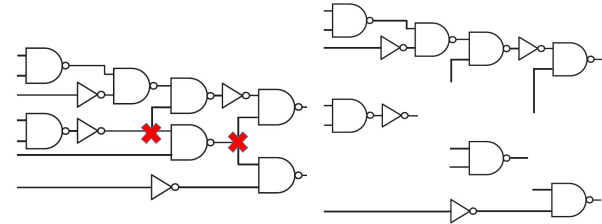
- Every node in the subject tree is covered by some library nodes
- The output of every library gate overlaps the input of the next library pattern



Technology Mapping via Tree Covering

Steps:

1. Subject DAG partitioned into a forest of trees



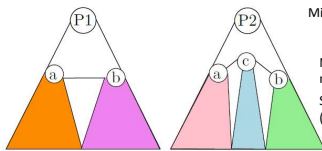
2. For each node in the subject tree, find all pattern trees in the library that have a match

Technology Mapping as a Tree Covering

Steps:

1. Subject DAG partitioned into a forest of trees
2. For each node in the subject tree, find all pattern trees in the library that have a match
3. Min-cost covering: pick a subset of matches to cover the subject tree with minimum cost

Minimum cost covering:



$$\text{MinCost}(\text{root}) = \min\{[\text{cost}(P1) + \text{MinCost}(a) + \text{MinCost}(b)], [\text{cost}(P2) + \text{MinCost}(a) + \text{MinCost}(b) + \text{MinCost}(c)]\}$$

Note: Covering algorithm revisits the same treenodes many times during recursion (duplicate computation!)
Sol: Keep a table storing MinCost value for each node (initialize with ∞)

Optimal cover for a tree consists of a match at the root of the tree + the optimal cover for the subtrees starting at each input of the match

Technology Mapping as a Tree Covering: Example

